

Física Computacional (106018C)

Taller [N.º 2] — Decaimiento radiactivo con NumPy y Matplotlib

Semana [N.º 3] — Lenguaje: Python

[Luis Arenas] — [21/09/2026]

1. Contexto físico. Se estudia el decaimiento radiactivo de una muestra de $N_0 = 10\,000$ núcleos con vida media $t_{1/2} = 5$ años; conocer $N(t)$ permite predecir cuánto material radiactivo queda en un instante dado, algo relevante en datación isotópica y en el manejo de residuos nucleares.

2. Método. Se retoma el modelo de decaimiento exponencial del taller anterior (antes en Fortran, ahora en Python), evaluando la función de forma vectorizada con NumPy (sin ciclo `for`) sobre un arreglo de tiempos generado con `numpy.linspace`:

$$N(t) = N_0 e^{-\lambda t}, \quad \lambda = \frac{\ln(2)}{t_{1/2}}$$

El tiempo necesario para que quede una fracción f de la muestra se obtiene invirtiendo la relación: $t = -\ln(f)/\lambda$.

3. Resultado. Se calculó el tiempo en que la muestra se reduce al 50 %, 25 % y 10 %, comparando el valor obtenido con la solución analítica (valor de referencia).

Caso	Valor obtenido	Valor de referencia	Error relativo
Queda 50 %	5.00 años	5.00 años ($t_{1/2}$)	0 %
Queda 25 %	10.00 años	10.00 años ($2t_{1/2}$)	0 %
Queda 10 %	16.61 años	16.61 años ($\ln(10)/\lambda$)	0 %

Verificación (PASS/FAIL): 3 de 3 casos de la celda de verificación pasaron (fracción remanente en $t = 0, 5, 10$ años comparada contra 1.00, 0.50 y 0.25, con tolerancia 10^{-3}).

4. Obstáculo o decisión de implementación. Al marcar sobre la gráfica los puntos de una, dos y tres vidas medias, una primera versión ubicaba los marcadores en $t = 1, 2, 3$ años; al revisar los resultados numéricos se notó que no correspondían a los puntos donde $N(t)$ cae a la mitad, cuarto y octavo de N_0 . Se corrigió ubicando los marcadores en $t = 5, 10, 15$ años, que sí coinciden con $t_{1/2}$, $2t_{1/2}$ y $3t_{1/2}$.

5. Conclusión. Se comprobó que la vectorización con NumPy reproduce exactamente los resultados analíticos sin necesidad de ciclos `for`, y que verificar visualmente los marcadores contra el valor esperado de $t_{1/2}$ es una forma sencilla de detectar errores de cómputo o de indexación de tiempo.

6. Uso de IA. Se utilizó inteligencia artificial como herramienta de apoyo durante el desarrollo del programa. Principalmente se empleó para depurar errores de compilación y lógica, proponer mejoras en la estructura y legibilidad del código, y verificar el cumplimiento de buenas prácticas de programación en Python. Las sugerencias obtenidas fueron revisadas y adaptadas al código desarrollado.

Código fuente: ver archivo adjunto [`decaimientoRad.py`].